

* Circular Dependency :-

It occurs when two or more beans depend on each other, creating a cycle that prevents the IOC container from successfully instantiating them.

- The Problem → If `OrderService` requires `PaymentService` in its constructor, and `PaymentService` requires `OrderService` in its constructor, Spring cannot decide which bean to create first, leading to a `'BeanCurrentlyInCreationException'`.
- Why it happens → The issue stems from Constructor Injection, where the dependency must be fully realized during object creation process.
- Common Misconception → While you could technically resolve this using Setters or Field Injection because these allow Spring to create an empty shell object first and inject dependencies later, this is generally considered a 'design flaw'.
- Best Practice → Instead of "solving" the cycle with technical tricks, you should refactor your code to adhere to Single Responsibility Principle. For example, move shared logic to a third component or ensure the dependency

graph is linear. (A depends on B, but B does not depend on A).

* Bean Scopes :-

In Spring, Bean Scope define the lifecycle and visibility of the objects managed by IOC Container. Choosing the correct scope is essential for performance and memory management.

→ Singleton Scope (Default) :

- Spring creates only one instance per bean definition.
- Every time you request the bean (via `context.getBean()` or dependency injection), Spring returns the same shared instance.
- This is highly efficient as it avoids unnecessary object creation.

→ Prototype Scope :

- A new Object Instance is created every time the bean is requested.
- Useful for stateful beans where you need an isolated instance for a specific task.

→ Web Scopes:

- Relevant for web applications. These include Request (created per HTTP request), Session (per user session), and Application (per Servlet context).

* Singleton beans are eagerly initialized by default at container startup. Prototype beans are lazily initialized - they are only created when actually requested.

* Bean Initialization:-

In Spring, how and when beans are created is controlled by the Initialization strategy. Understanding this helps optimize application startup and memory usage.

→ Eager Initialization (Default):

- By default, Spring creates all singleton beans at application startup.
- This ensures any issues with bean creation are caught immediately when the context starts.

→ Lazy Initialization (@Lazy):

- When you mark a bean with @Lazy, Spring skips creating the object

during startup.

- The bean is only initialized the first time it is requested or injected into another bean.
- This can significantly improve startup time for large applications with many beans that aren't needed right away.

→ Handling Circular Dependency with @Lazy:

- @Lazy can be used to resolve circular dependencies by injecting a 'proxy' instead of the actual object during initialization.
- This breaks the "chicken & egg" creation loop: Spring creates an "empty" shell (the proxy) for one of the beans, allowing the other to finish creating, and injects the real object later when the dependency is actually accessed.

→ Global Lazy Initialization:

- You can set "spring.main.lazyInitialization = true" in your application properties to make all beans lazy by default, though this is a global configuration choice.